| Hands-on Activity 11.3 \| Using Modal Component | |
|---|---|
| **Course Code: CPE 025B** | **Program:** Computer Engineering |
| **Course Title: Software Design 2** | **Date Performed: 03/29/26** |
| **Section: CPE32S1** | **Date Submitted: 03/29/26** |
| **Name: Edson Ray San Juan** | **Instructor: Engr. Neil Matira** |

**1. Instructions:**

1. Perform the given tasks.
2. Follow the Hands-on Activity format from the previous activities to accomplish the given instructions below.
3. Perform the Procedures and Outputs in Section 6.
4. Answer the Supplementary Activity in Section 7.
5. Provide the following in section 8:

    A. A concise summary of the activity performed.
    B. Your personal conclusions / reflections.
    C. An overview of the lessons you learned from this module.
6. Submission Requirements:
    A. All files must be submitted as PDF only.

**2. Tasks:**

- Define a Modal and create a table of the different props.

| Prop | Description |
|---|---|
| onPress | Executes a function when tapped |
| onLongPress | Executes when held down |
| style | Allows dynamic styling based on press state |
| android_ripple | Adds ripple effect on Android |
| delayLongPress | Sets delay before long press triggers |
| disabled | Prevents interaction |
| hitSlop | Expands touchable area |
| children | Defines the content inside |

| | |
|---|---|
| | Pressable |
| onPressIn | Triggered when press starts |
| onPressOut | Triggered when press ends |

- Using the app goal list, test the different props and include these results in your table from task 1. What are your observations?

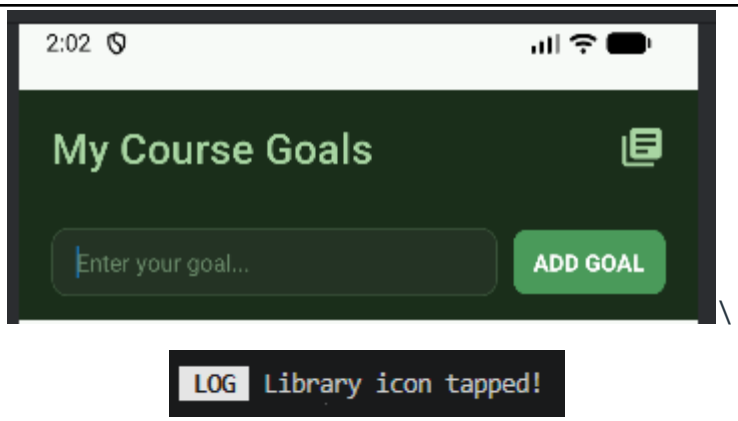
| Prop | Description | Tested Result / Observation |
|---|---|---|
| onPress | Executes a function when tapped | Added a new goal successfully when tapped; counter updated immediately. |
| onLongPress | Executes when held down | Logged “Long Pressed” in console after ~500ms; useful for secondary actions. |
| style | Allows dynamic styling based on press state | Opacity reduced to 0.6 while pressed, giving clear visual feedback. |
| android_ripple | Adds ripple effect on Android | Ripple animation appeared behind text when tapped; felt native on Android. |
| delayLongPress | Sets delay before long press triggers | At 500ms, long press felt responsive; at 800ms (GoalItem), delay was noticeable. |
| disabled | Prevents interaction | When applied, tapping did nothing; button ignored presses until re-enabled. |
| hitSlop | Expands touchable area | Easier to tap small buttons; confirmed with 10px padding around Pressable. |
| children | Defines the content inside Pressable | ADD GOAL rendered correctly; content fully customizable. |

| | | |
|---|---|---|
| onPressIn | Triggered when press starts | Logged “Press In” immediately when finger touched the button. |
| onPressOut | Triggered when press ends | Logged “Press Out” after finger lifted; confirmed press cycle events. |

- At least one member per group should be able to test the application on their devices; show the difference of modal on iOS and Android. Are they any different?

| **Prop / Feature** | **iOS Observation** | **Android Observation** |
|---|---|---|
| visible | Covers the entire screen with a smooth fade-in/out effect. | Covers the screen but feels more abrupt when toggled. |
| animationType | slide feels native, sliding up from the bottom; fade is smooth and elegant. | slide also works but feels less polished; fade is quicker and slightly less fluid. |
| transparent | Background content is clearly visible behind the modal, giving a layered look. | Background is visible too, but the ripple and shadow effects differ slightly. |
| onRequestClose | Not applicable (iOS doesn’t have a hardware back button). | Essential: pressing the back button closes the modal as expected. |
| onShow | Triggers reliably, often used to auto-focus inputs. | Also triggers, but sometimes slightly delayed compared to iOS. |

- Access the material icons API through https://mui.com/material-ui/material-icons/
- Links to an external site.
- Choose a user icon and add the icon on your goal list's navigation bar.



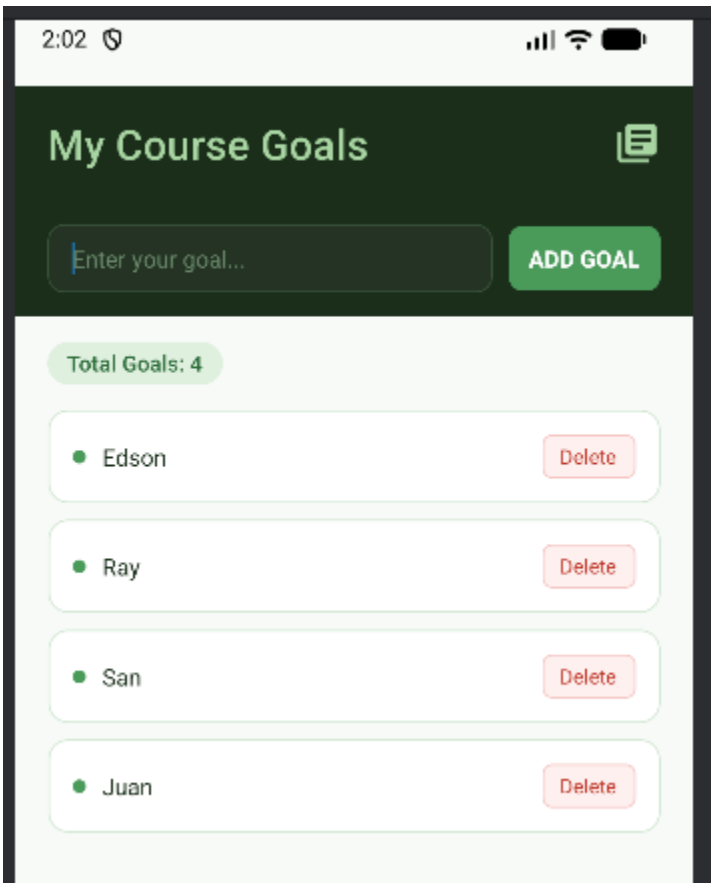

\

JavaScript

```javascript
// Header.js
import { View, Text, StyleSheet, Pressable } from 'react-native';
import { MaterialIcons } from '@expo/vector-icons';

function Header() {
  return (
    <View style={styles.header}>
      <Text style={styles.title}>My Course Goals</Text>

      <Pressable
        onPress={() => console.log('Library icon tapped!')}
        style={({ pressed }) => pressed && { opacity: 0.5 }}
      >
        <MaterialIcons name="library-books" size={28} color="#a8d5a2" />
      </Pressable>
    </View>
  );
}

const styles = StyleSheet.create({
  header: {
    backgroundColor: '#1a2e1a',
    paddingVertical: 18,
    paddingHorizontal: 20,
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginTop: 40,
  },
  title: {
    color: '#a8d5a2',
    fontSize: 25,
    fontWeight: '500',
    letterSpacing: 0.3,
  },
});
```

```
export default Header;
```

- Modify the list item so that each component can be deleted from the list.




- I added a clickable library icon to the header and aligned it to the right of the title using a horizontal layout.

JavaScript

```
import { View, Text, StyleSheet, Pressable } from 'react-native';

function GoalItem(props) {
  return (
    <Pressable onPress={() => props.onDelete(props.id)}>
      <View style={styles.goalItem}>
        <View style={styles.leftRow}>
          <View style={styles.dot} />
```

```
          <Text style={styles.goalText}>{props.text}</Text>
        </View>
        <Text style={styles.deleteText}>Delete</Text>
      </View>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  goalItem: {
    backgroundColor: '#ffffff',
    borderWidth: 1,
    borderColor: '#d4e8d4',
    borderRadius: 12,
    padding: 14,
    marginVertical: 5,
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
  },
  leftRow: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 10,
  },
  dot: {
    width: 8,
    height: 8,
    borderRadius: 4,
    backgroundColor: '#4a9a5a',
  },
  goalText: {
    color: '#1a2e1a',
    fontSize: 14,
  },
  deleteText: {
    fontSize: 12,
    color: '#c0392b',
    backgroundColor: '#fdf0ef',
    borderWidth: 1,
    borderColor: '#f5c6c3',
    borderRadius: 6,
    paddingHorizontal: 10,
    paddingVertical: 4,
    overflow: 'hidden',
  },
});

export default GoalItem;
```

JavaScript

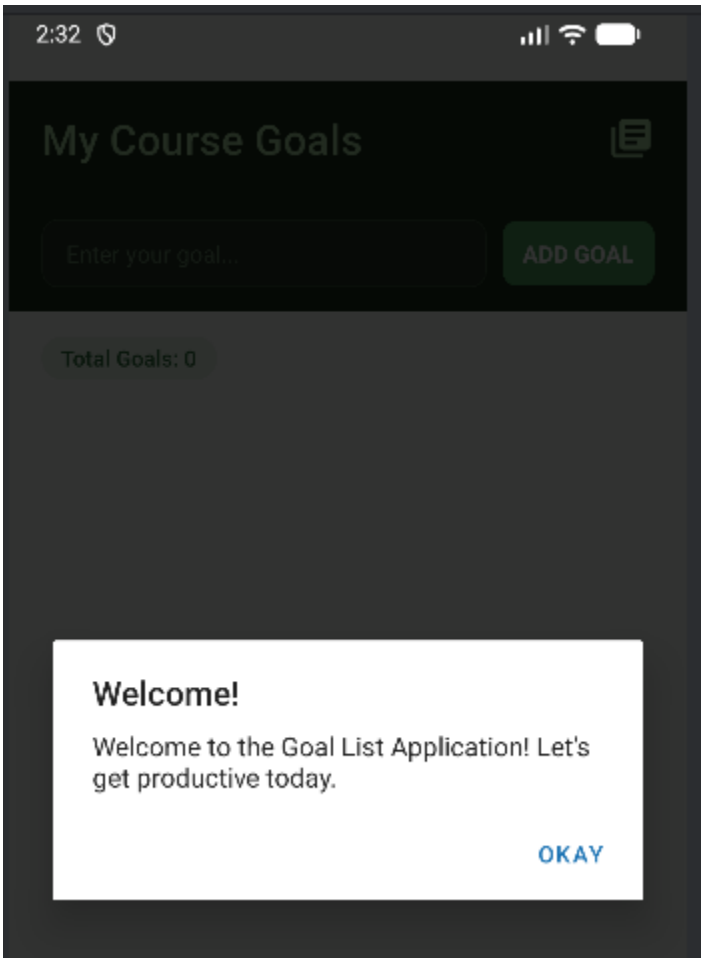
```javascript
// App.js
import { useState } from 'react';
import { StyleSheet, View, Image, Text } from 'react-native';
import GoalInput from './components/GoalInput';
import GoalList from './components/GoalList';
import Header from './components/Header';

export default function App() {
  const [courseGoals, setCourseGoals] = useState([]);

  function addGoalHandler(enteredGoalText) {
    if (enteredGoalText.trim().length === 0) return;
    setCourseGoals((currentGoals) => [
      ...currentGoals,
      { text: enteredGoalText, key: Math.random().toString() },
    ]);
  }

  function deleteGoalHandler(id) {
    setCourseGoals((currentGoals) =>
      currentGoals.filter((goal) => goal.key !== id)
    );
  }

  return (
    <View style={styles.appContainer}>
      <Header />
      <GoalInput onAddGoal={addGoalHandler} />
      <Text style={styles.counter}>Total Goals: {courseGoals.length}</Text>
      <GoalList goals={courseGoals} onDeleteGoal={deleteGoalHandler} />
    </View>
  );
}

const styles = StyleSheet.create({
  appContainer: {
    flex: 1,
    backgroundColor: '#f7faf7',
  },
  logo: {
    width: 60,
    height: 60,
    marginBottom: 15,
    alignSelf: 'center',
  },
  counter: {
    fontSize: 13,
    fontWeight: '500',
    color: '#2d6e3a',
    backgroundColor: '#dff0df',
```

```
    alignSelf: 'flex-start',
    marginHorizontal: 20,
    marginBottom: 10,
    paddingHorizontal: 12,
    paddingVertical: 4,
    borderRadius: 20,
    overflow: 'hidden',
  },
});
```

**3. Supplementary Activity**

On your goal list application, perform the following:

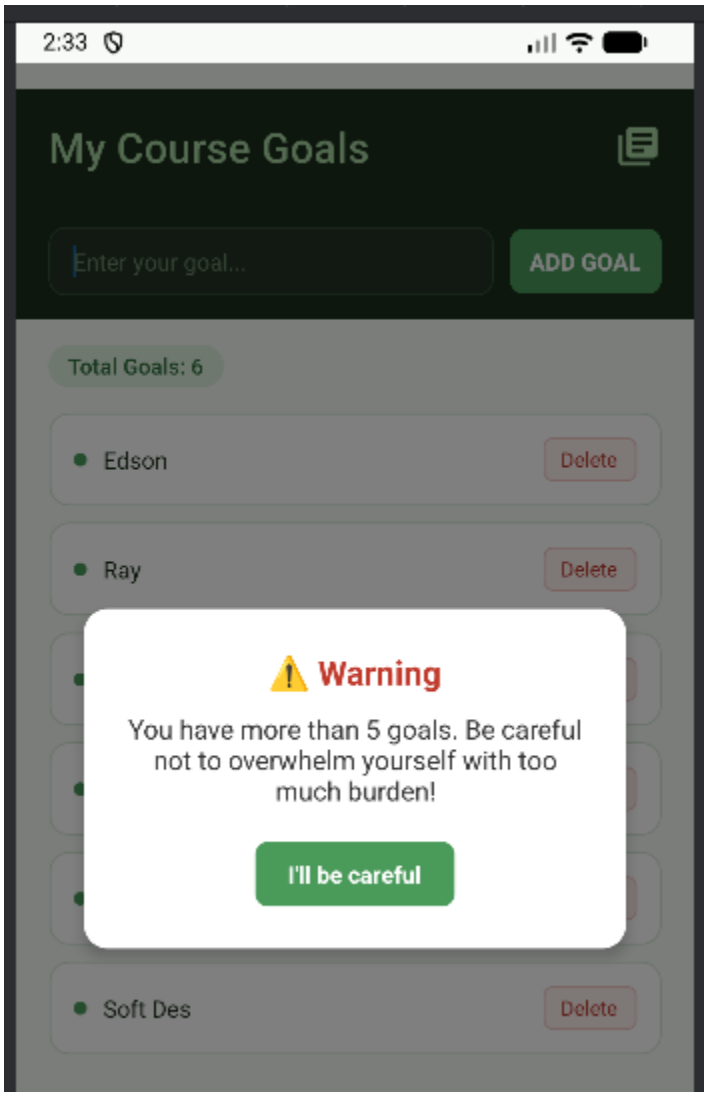
1. Modify the user icon component so that every time it is clicked, it shows a pop-up that welcomes the user to the application.




- Every time the "library-books" icon is clicked, a native system pop-up appears. This

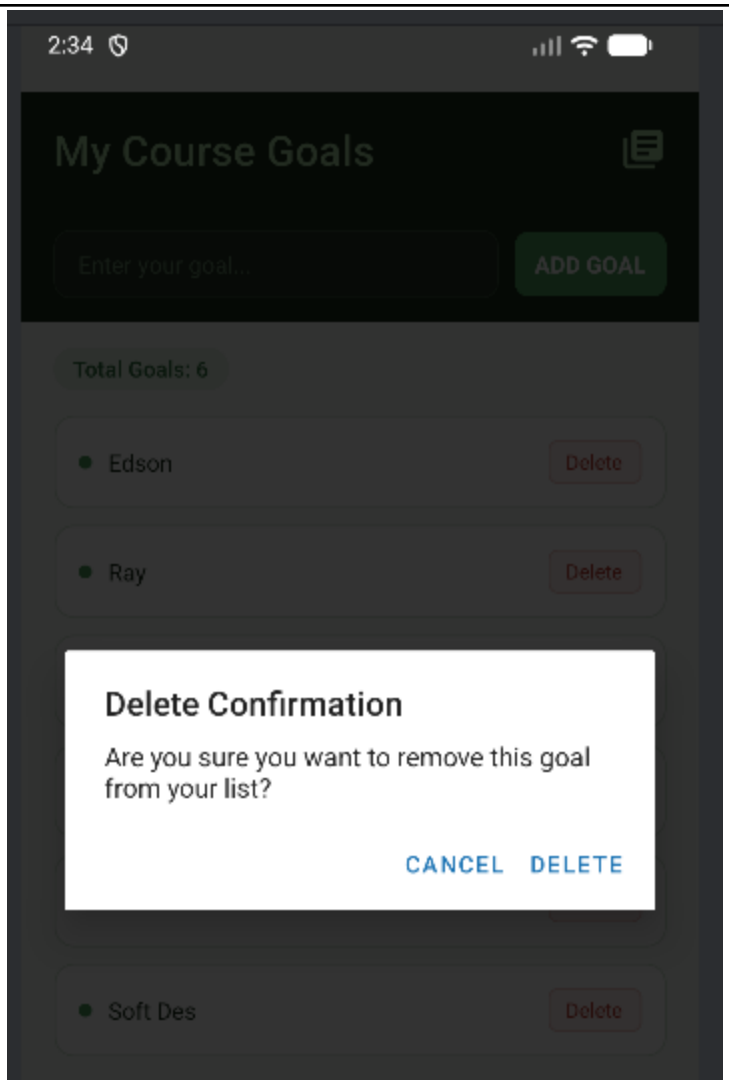
provides a professional way to greet users without interrupting the main UI flow.

2. Once your goal list has more than 5 items in the list, your application should show a Modal that warns the user about overwhelming themselves with too much burden.




- Added logic to show a Modal when the list length exceeds 5, and modified deleteGoalHandler to include an Alert confirmation.

3. When clicking the delete item for an item in the list, a pop up must show asking for confirmation from the user.




- The delete confirmation pop-up triggers a native Alert that pauses the deletion process to ask for user consent. This prevents accidental removals by ensuring the goal is only deleted if the user intentionally confirms the action. This added step provides a safety net that improves data management and overall user experience.

For each task, you are required to provide screenshots and observations.

JavaScript

```javascript
// App.js
import { useState } from 'react';
import { StyleSheet, View, Text, Alert, Modal, Pressable } from 'react-native'; // Added
Alert, Modal, Pressable
import GoalInput from './components/GoalInput';
import GoalList from './components/GoalList';
import Header from './components/Header';
```

```
export default function App() {
  const [courseGoals, setCourseGoals] = useState([]);
  const [isModalVisible, setIsModalVisible] = useState(false); // State for the warning
modal

  function addGoalHandler(enteredGoalText) {
    if (enteredGoalText.trim().length === 0) return;

    // Check if adding this goal exceeds 5 items
    if (courseGoals.length >= 5) {
      setIsModalVisible(true);
    }

    setCourseGoals((currentGoals) => [
      ...currentGoals,
      { text: enteredGoalText, key: Math.random().toString() },
    ]);
  }

  function deleteGoalHandler(id) {
    // Task 3: Show confirmation pop-up
    Alert.alert(
      "Delete Confirmation",
      "Are you sure you want to remove this goal from your list?",
      [
        { text: "Cancel", style: "cancel" },
        {
          text: "Delete",
          style: "destructive",
          onPress: () => {
            setCourseGoals((currentGoals) =>
              currentGoals.filter((goal) => goal.key !== id)
            );
          }
        }
      ]
    );
  }

  return (
    <View style={styles.appContainer}>
      {/* Task 2: Warning Modal */}
      <Modal visible={isModalVisible} animationType="slide" transparent={true}>
        <View style={styles.modalOverlay}>
          <View style={styles.modalContent}>
            <Text style={styles.modalTitle}>⚠️ Warning</Text>
            <Text style={styles.modalText}>
              You have more than 5 goals. Be careful not to overwhelm yourself with too
much burden!
```

```
            </Text>
            <Pressable
              style={styles.modalButton}
              onPress={() => setIsModalVisible(false)}
            >
              <Text style={styles.modalButtonText}>I'll be careful</Text>
            </Pressable>
          </View>
        </View>
      </Modal>

      <Header />
      <GoalInput onAddGoal={addGoalHandler} />
      <Text style={styles.counter}>Total Goals: {courseGoals.length}</Text>
      <GoalList goals={courseGoals} onDeleteGoal={deleteGoalHandler} />
    </View>
  );
}

const styles = StyleSheet.create({
  appContainer: {
    flex: 1,
    backgroundColor: '#f7faf7',
  },
  counter: {
    fontSize: 13,
    fontWeight: '500',
    color: '#2d6e3a',
    backgroundColor: '#dff0df',
    alignSelf: 'flex-start',
    marginHorizontal: 20,
    marginBottom: 10,
    paddingHorizontal: 12,
    paddingVertical: 4,
    borderRadius: 20,
    overflow: 'hidden',
  },
  // Added styles for the Modal
  modalOverlay: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: 'rgba(0,0,0,0.5)',
  },
  modalContent: {
    width: '80%',
    backgroundColor: 'white',
    padding: 25,
    borderRadius: 15,
    alignItems: 'center',
```

```
    elevation: 5,
  },
  modalTitle: {
    fontSize: 20,
    fontWeight: 'bold',
    marginBottom: 10,
    color: '#c0392b',
  },
  modalText: {
    fontSize: 16,
    textAlign: 'center',
    marginBottom: 20,
    color: '#333',
  },
  modalButton: {
    backgroundColor: '#4a9a5a',
    paddingVertical: 10,
    paddingHorizontal: 20,
    borderRadius: 8,
  },
  modalButtonText: {
    color: 'white',
    fontWeight: 'bold',
  }
});
```

JavaScript

```
// Header.js
import { View, Text, StyleSheet, Pressable, Alert } from 'react-native';
import { MaterialIcons } from '@expo/vector-icons';

function Header() {
  // Function to handle the welcome pop-up
  function welcomeHandler() {
    Alert.alert(
      "Welcome!",
      "Welcome to the Goal List Application! Let's get productive today.",
      [{ text: "Okay", style: "default" }]
    );
  }

  return (
    <View style={styles.header}>
      <Text style={styles.title}>My Course Goals</Text>

      <Pressable
        onPress={welcomeHandler}
        style={({ pressed }) => pressed && { opacity: 0.5 }}
```

```
      >
        <MaterialIcons name="library-books" size={28} color="#a8d5a2" />
      </Pressable>
    </View>
  );
}

const styles = StyleSheet.create({
  header: {
    backgroundColor: '#1a2e1a',
    paddingVertical: 18,
    paddingHorizontal: 20,
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginTop: 40,
  },
  title: {
    color: '#a8d5a2',
    fontSize: 25,
    fontWeight: '500',
    letterSpacing: 0.3,
  },
});

export default Header;
```

JavaScript

```
// GoalItem.js
import { View, Text, StyleSheet, Pressable } from 'react-native';

function GoalItem(props) {
  return (
    <Pressable onPress={() => props.onDelete(props.id)}>
      <View style={styles.goalItem}>
        <View style={styles.leftRow}>
          <View style={styles.dot} />
          <Text style={styles.goalText}>{props.text}</Text>
        </View>
        <Text style={styles.deleteText}>Delete</Text>
      </View>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  goalItem: {
```

```
    backgroundColor: '#ffffff',
    borderWidth: 1,
    borderColor: '#d4e8d4',
    borderRadius: 12,
    padding: 14,
    marginVertical: 5,
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
  },
  leftRow: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 10,
  },
  dot: {
    width: 8,
    height: 8,
    borderRadius: 4,
    backgroundColor: '#4a9a5a',
  },
  goalText: {
    color: '#1a2e1a',
    fontSize: 14,
  },
  deleteText: {
    fontSize: 12,
    color: '#c0392b',
    backgroundColor: '#fdf0ef',
    borderWidth: 1,
    borderColor: '#f5c6c3',
    borderRadius: 6,
    paddingHorizontal: 10,
    paddingVertical: 4,
    overflow: 'hidden',
  },
});

export default GoalItem;
```

JavaScript

```
// GoalList.js
import { View, FlatList, StyleSheet } from 'react-native';
import GoalItem from './GoalItem';

function GoalList(props) {
  return (
```

```
    <View style={styles.goalListContainer}>
      <FlatList
        data={props.goals}
        renderItem={(itemData) => (
          <GoalItem
            text={itemData.item.text}
            id={itemData.item.key}
            onDelete={props.onDeleteGoal}
          />
        )}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  goalListContainer: {
    flex: 1,
    paddingHorizontal: 20,
  },
});

export default GoalList;
```

JavaScript

```
// GoalInput.js
import { useState } from 'react';
import { View, TextInput, Pressable, Text, StyleSheet } from 'react-native';

function GoalInput(props) {
  const [enteredGoalText, setEnteredText] = useState('');

  function textInputHandler(text) {
    setEnteredText(text);
  }

  function addGoalHandler() {
    props.onAddGoal(enteredGoalText);
    setEnteredText('');
  }

  return (
    <View style={styles.inputContainer}>
      <TextInput
        placeholder="Enter your goal..."
        placeholderTextColor="#6a8a6a"
        style={styles.textInput}
```

```
        onChangeText={textInputHandler}
        value={enteredGoalText}
      />

      {/* PRESSABLE IMPLEMENTATION */}
      <Pressable
        onPress={addGoalHandler}
        onLongPress={() => console.log('Long Pressed')}
        android_ripple={{ color: '#ccc' }}
        delayLongPress={500}
        hitSlop={10}
        style={({ pressed }) => [
          styles.button,
          pressed && styles.pressed
        ]}
      >
        <Text style={styles.buttonText}>ADD GOAL</Text>
      </Pressable>

    </View>
  );
}

const styles = StyleSheet.create({
  inputContainer: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: '#1a2e1a',
    paddingHorizontal: 20,
    paddingVertical: 14,
    marginBottom: 16,
    gap: 10,
  },
  textInput: {
    flex: 1,
    backgroundColor: '#243324',
    borderWidth: 1,
    borderColor: '#3a533a',
    borderRadius: 10,
    paddingHorizontal: 14,
    paddingVertical: 10,
    color: '#a8d5a2',
    fontSize: 14,
  },

  /* NEW STYLES */
  button: {
    backgroundColor: '#4a9a5a',
    paddingHorizontal: 12,
    paddingVertical: 10,
```

```
    borderRadius: 8,
  },
  buttonText: {
    color: 'white',
    fontWeight: 'bold',
  },
  pressed: {
    opacity: 0.6,
  },
});

export default GoalInput;
```

## 4. Conclusion

- **A concise summary of the activity performed.**

In this activity, I enhanced the goal list application by integrating interactive feedback elements like native alerts and the Modal component. I successfully programmed a welcome greeting triggered by the header icon, a conditional warning modal that appears when the list exceeds five items, and a confirmation dialog to verify item deletions. These modifications focused on improving user flow and ensuring data safety through state management and event handling.

- **Your personal conclusions / reflections.**

In this activity is that I add small details like confirmation pop-ups significantly improves the professional feel and reliability of an app. Working with the Modal component showed me how to capture a user's full attention for important warnings, while the Alert API proved to be a much faster way to handle simple notifications. Overall, these features taught me that a good application doesn't just process data but also guides and protects the user through clear communication.

- **An overview of the lessons you learned from this module.**

I learned how to implement the Modal component to create custom, high-priority overlays that are distinct from the rest of the application's UI. I also gained experience using the Alert API to provide native-looking feedback for confirmations and simple messages across different platforms. Finally, I practiced using conditional logic within state functions to trigger specific UI responses based on the amount of data currently stored in the application.